

Pensamiento Computacional para Jóvenes en la Práctica

Irene Lee ■ Fred Martin ■ Jill Denner ■ Bob Coulter ■ Walter Allan
Jeri Erickson ■ Joyce Malyn-Smith ■ Linda Werner

Trad: Cristián Rizzi Iribarren

El **Pensamiento Computacional (PeCo)** ha sido descrito como el uso de la abstracción, la automatización y el análisis en la resolución de problemas [3]. Examinamos cómo estas maneras de pensar toman forma para los jóvenes de secundaria y preparatoria en un conjunto de programas apoyados por NSF¹. Discutimos oportunidades y desafíos en contextos escolares y extraescolares. Con base en estas observaciones, presentamos un marco de "usar-modificar-crear", que representa tres fases de la actividad cognitiva y práctica de los estudiantes en el pensamiento computacional. Recomendamos una inversión continua en el desarrollo de entornos de aprendizaje ricos en pensamiento computacional, en educadores que puedan facilitar su uso y en la investigación sobre el valor más amplio del pensamiento computacional.

1 INTRODUCCIÓN

El pensamiento computacional (PeCo) es un término acuñado por Jeannette Wing [11] para describir un conjunto de habilidades, hábitos y enfoques de pensamiento que son parte integral de la resolución de problemas complejos utilizando una computadora y que se aplican ampliamente en la sociedad de la información. El pensamiento computacional se basa en los conceptos que son fundamentales para las ciencias de la computación e involucra el procesamiento de información y tareas de manera sistemática y eficiente. El PeCo implica definir, comprender y resolver problemas, razonar en múltiples niveles de abstracción,

comprender y aplicar la automatización y analizar la idoneidad de las abstracciones realizadas. El pensamiento computacional comparte elementos con varios otros tipos de pensamiento, como el pensamiento algorítmico, el pensamiento ingenieril, el pensamiento de diseño y el pensamiento matemático. Como tal, el pensamiento computacional se basa en un rico legado de marcos relacionados, ya que amplía las habilidades de pensamiento anteriores.

Este documento tiene como objetivo ayudar a los educadores de informática y STEM (ciencia, tecnología, ingeniería y matemáticas) a comprender el pensamiento computacional (cómo se ve "en la práctica", cómo se conecta con un plan de estudios existente y cómo fomentar el pensamiento computacional en los jóvenes de hoy) mediante ejemplos valiosos de los programas de Experiencias Tecnológicas Innovadoras para Estudiantes y Docentes (ITEST), Academias para Jóvenes Científicos (AYS) e Investigación y Evaluación en Educación en Ciencias e Ingeniería (REESE) financiadas por la NSF. Los ejemplos proporcionan una lente a través de la cual se pueden considerar las implicancias para el aprendizaje y la enseñanza del pensamiento computacional en los grados de Sala de 5 en el Nivel Inicial hasta el último año de la escuela secundaria.

Las preguntas clave incluyen:

- *¿Cómo es el pensamiento computacional para jóvenes en la práctica?*
- *¿Cómo podemos apoyar el crecimiento del pensamiento computacional, tanto dentro como fuera del ámbito?*

Los ejemplos y recomendaciones presentados en este documento fueron recopilados por el grupo de trabajo de ITEST sobre pensamiento computacional. Todos los autores son miembros de esta comunidad en virtud de su participación en

¹ NSF: National Science Foundation: Organismo estatal que financia proyectos de educación en ciencias

programas ITEST actuales o anteriores. Este trabajo tiene como objetivo complementar la serie de talleres "Pensamiento computacional para todos" de las Academias Nacionales y el trabajo que actualmente están llevando a cabo la Computer Science Teachers Association (CSTA) y la International Society for Technology in Education (ISTE) como parte del Computational Thinking Thought Leaders Program, y promover la discusión presentando ejemplos de pensamiento computacional en acción dentro de programas para jóvenes tanto en entornos de educación formal como no formal.

2 PENSAMIENTO COMPUTACIONAL PARA JÓVENES EN LA PRÁCTICA

En este artículo, respondemos a varios requerimientos recientes para describir el PeCo entre los jóvenes e identificar estrategias para integrar el PeCo en entornos escolares [4] [5] [7]. Aplicamos y construimos sobre las descripciones existentes del pensamiento computacional, que se han basado en pensar como un científico de la computación en la universidad y más allá. Específicamente, ofrecemos ejemplos de cómo se ve el pensamiento computacional entre los jóvenes de una variedad de orígenes culturales y socioeconómicos, tanto dentro como fuera de la escuela. Los ejemplos se extraen de tres dominios: modelización y simulación; robótica; y diseño y desarrollo de juegos. En todos estos dominios, hemos identificado puntos en común con la naturaleza del pensamiento computacional en los jóvenes.

Encontramos que los términos de abstracción, automatización y análisis [3] son útiles para comprender cómo los jóvenes pueden usar el PeCo para abordar problemas nuevos. La abstracción es "el proceso de generalizar a partir de instancias específicas". En la resolución de problemas, la abstracción puede tomar la forma de reducir un problema a lo que se cree que son sus elementos esenciales.

La abstracción también se define comúnmente como la captura de características o acciones comunes en un conjunto que pueden usarse para representar todas las demás instancias. La automatización es un proceso que ahorra trabajo en el que se instruye a una computadora

para que ejecute un conjunto de tareas repetitivas de manera rápida y eficiente en comparación con el poder de procesamiento de un ser humano. En este sentido, los programas de computadora son "automatizaciones de abstracciones". El análisis es una práctica reflexiva que se refiere a la validación de si las abstracciones realizadas fueron correctas. Uno podría preguntarse "¿Se hicieron las suposiciones correctas al reducir el problema a lo esencial?", "¿Se dejaron de lado los factores importantes?" o "¿La implementación de la abstracción o la automatización fue defectuosa?" La Tabla 1 proporciona un resumen de estos dominios:

TABLA 1: EJEMPLOS DE PENSAMIENTO COMPUTACIONAL EN TRES DOMINIOS

	Abstracción	Automación	Análisis
Modelado y simulación	Selección de características del mundo real para incorporarlas a un modelo	Pasos en el tiempo usando un modelo como banco experimental de pruebas	¿Son correctas las abstracciones realizadas? ¿El modelo refleja la realidad?
Robótica	Diseñar un robot que reaccione a un conjunto de condiciones.	El programa verifica los sensores para monitorear las condiciones.	¿Hay situaciones que no se tomaron en cuenta?
Diseño y desarrollo de juegos	Los juegos se resumen en un conjunto de escenas que contienen personajes.	El juego responde a las acciones del usuario.	¿Los elementos incorporados hacen que el juego sea divertido?

En las siguientes secciones, usamos ejemplos de tres programas extraescolares para jóvenes (OST) para ilustrar cómo se ven los tres aspectos del PeCo en la práctica, en cada uno de los tres dominios. Cada uno de estos programas ofrece oportunidades para que los estudiantes de secundaria y preparatoria se involucren en el pensamiento computacional. Los estudiantes presentan niveles desparejos en cuanto a su experiencia en el manejo de la computadora, incluidos estudiantes con un inglés limitado y sin computadora en sus hogares, así como estudiantes

que han crecido jugando con la tecnología. La naturaleza práctica y orientada a los jóvenes de los programas, está pensada para permitir que los estudiantes de todos los niveles se involucren en el pensamiento computacional.

2.1. Modelización y simulación

El primer dominio que consideraremos es la modelización y la simulación. Dave Moursund [6] sugiere que “la idea subyacente en el pensamiento computacional es desarrollar modelos y simulaciones de problemas que uno está tratando de estudiar y resolver”. En el Proyecto GUTS (Crecer Pensando Científicamente), los estudiantes de secundaria participan activamente en el pensamiento computacional mientras diseñan e implementan modelos de relevancia local y luego usan los modelos para ejecutar simulaciones. Los estudiantes utilizaron el proceso de abstracción para reducir el problema a algo que pudiera implementarse en la computadora usando StarLogo TNG, una herramienta de modelización basada en agentes. Las restricciones impuestas por este entorno de modelización incluyen un límite superior en la cantidad de agentes (4076) y un límite en el tamaño del entorno (101 por 101 celdas/baldosas/cuadrados).

Dentro de estos parámetros, los estudiantes diseñaron y crearon modelos como bancos de pruebas para responder preguntas sobre problemas del mundo real. Por ejemplo, como parte de la unidad del Proyecto GUTS sobre epidemiología, un grupo de estudiantes quería saber si una enfermedad se propagaría por toda la población escolar dada la distribución de la escuela, el número de estudiantes, el movimiento de los estudiantes, la virulencia de la enfermedad y el número de estudiantes inicialmente infectados. Ver la Figura 1.



Figura 1: Miembros del club GUTS que crean modelos de ecosistemas en Chicago.

Al mapear esta pregunta y este escenario en un modelo basado en agentes, los agentes se utilizaron como abstracciones o representaciones simplificadas de los estudiantes y el número de agentes coincidió con el número de estudiantes en su escuela. Se les dio a los agentes comportamientos de movimiento que eran abstracciones de moverse de un aula en aula, y se tomaron decisiones sobre qué características de la escuela eran importantes a tener en cuenta antes de crear un modelo virtual en 3D del edificio de la escuela. Por ejemplo, los estudiantes decidieron que era importante recrear el número y la ubicación de los pasillos y las puertas de la escuela.

Además, los estudiantes modelizaron las características de contagio: la frecuencia con la que el contacto entre estudiantes propaga la enfermedad de uno a otro y cuántos estudiantes se infectaron inicialmente.

Para hacer del modelo un banco de pruebas capaz de ejecutar experimentos, el modelo incluyó controles deslizantes en la interfaz para controlar variables individuales. Un control deslizante permitía configurar el número de agentes inicialmente infectados y otro controlaba el nivel de virulencia del patógeno. Ver Figura 2.



Figura 2: Personalización de los estudiantes del modelo de contagio para reflejar el diseño de la escuela.

La automatización se utilizó de varias formas. El “programa” en sí automatizó el “paso a través” o el avance de la simulación mediante el uso de un ciclo de ejecución que actualizaba el estado, la ubicación y el color de cada agente (que representaba si estaba enfermo o saludable) en cada unidad de tiempo. Debido a que los modelos basados en agentes involucran aleatoriedad (por ejemplo, la ubicación inicial de los individuos infectados se elige al azar), los resultados hablan de probabilidades en lugar de predicciones. Se utilizó la automatización para realizar múltiples “ejecuciones” del experimento con la misma

configuración de parámetros a fin de alcanzar las probabilidades de ciertos resultados. Una vez que se ejecutaron las simulaciones y se recopilaron los datos sobre el número de personas infectadas después de haber transcurrido un número fijo de unidades de tiempo, los estudiantes reflexionaron sobre los resultados. En algunos casos, los estudiantes pudieron analizar sus modelos, las suposiciones y abstracciones hechas, comparando los datos generados por el modelo con los datos recolectados dentro de sus escuelas. Por ejemplo, un grupo comparó los datos generados con su modelo que simulaba la propagación de la gripe porcina con los registros de asistencia / absentismo recopilados durante un período de tiempo en el que se sabía que la gripe porcina circulaba dentro de su escuela. Un análisis de este tipo puede llevar a reconsiderar qué factores incluir en un modelo y volver al comienzo del proceso descrito anteriormente.

2.2. Robótica

Un segundo dominio que promueve el pensamiento computacional con los estudiantes preuniversitarios es la robótica. En un proyecto de robótica, los estudiantes de programación diseñan y programan robots y otros dispositivos físicos con código incrustado. Deben pensar cómo interactuará el agente robótico dentro de su mundo, basándose en factores como los valores de sus sensores y los efectos de sus actuadores. Al hacer esto, el estudiante elige cómo su programación conectará estos procesos para lograr los resultados deseados.

En el proyecto iCODE (Comunidad de Ingenieros de Diseño de Internet), los estudiantes de secundaria y preparatoria completan una variedad de proyectos basados en microcontroladores, comenzando con un proyecto simple con luces intermitentes programables, pasando por un juego de memoria musical, hasta robots completamente autónomos (autocontrolados) que participan en un concurso. Ver la Figura 3.



Figura 3: Dos estudiantes de iCODE muestran su robot Sumo.

La abstracción tiene lugar cuando los estudiantes diseñan robots para reaccionar a un conjunto limitado de condiciones que pueden encontrarse en el mundo real.

Los estudiantes piensan en cómo sensor el mundo y cómo esos estímulos pueden abstraerse como valores numéricos o resultados verdadero-falso dentro del programa de control. La automatización ocurre cuando los programas de los estudiantes son ejecutados por el dispositivo informático integrado. Los estudiantes realizan análisis cuando deciden si el robot funcionó o no como se esperaba en el entorno del mundo real. Si el robot "se porta mal", puede significar que la implementación de su idea de control es defectuosa o que se encontraron condiciones que no se tomaron en cuenta durante la fase de abstracción.

2.3 Diseño y desarrollo de juegos

Un tercer dominio en el que tiene lugar el pensamiento computacional es el diseño y desarrollo de juegos de computadora. En el programa extracurricular iGame, los estudiantes de secundaria se involucran en el pensamiento computacional diseñando, programando y probando juegos de computadora de su elección usando Storytelling Alice (Alice). Alice, al igual que muchos otros lenguajes de programación, permite a los estudiantes crear sus propias abstracciones.

Debido a que Alice es un entorno de programación que permite la creación de

animaciones 3D, los estudiantes pueden probar abstracciones altamente complejas de forma rápida y precisa. Para crear su juego, los estudiantes construyen un grupo de escenas, donde cada escena contiene personajes y cada personaje tiene comportamientos. Los estudiantes eligen entre una variedad de atributos y comportamientos de los personajes, seleccionando solo aquellos detalles apropiados para el mundo virtual que están creando.

Los estudiantes pueden definir nuevos métodos que representen comportamientos no integrados en Alice. En estos métodos, los estudiantes pueden colocar una combinación de comportamientos existentes y cambios en los atributos del carácter. En iGame, muchos métodos creados por estudiantes eran combinaciones simples de comandos secuenciales, pero la creación de métodos a menudo requiere una comprensión de los condicionales, la iteración y la ejecución secuencial y paralela. Por ejemplo, en el juego Laberinto de la Tortuga, un estudiante programó un personaje para bailar usando una serie de movimientos paralelos y vocalizaciones.

Los juegos a menudo requieren varios personajes similares para realizar la misma acción. Los estudiantes pueden "ampliar" su abstracción creando una estructura de datos de lista que contenga estos caracteres similares. Luego, pueden programar comportamientos similares para estos personajes mediante el uso de instrucciones especiales que recorren una estructura de datos en forma de lista. Por ejemplo, en el juego Invasión Zombie, un estudiante programó un grupo de zombis para que esperaran y luego comenzaran a moverse al mismo tiempo. A medida que el jugador hace clic en cada uno, lo está programando para desaparecer. Sin embargo, la mayoría de los programadores de juegos de iGame optan por repetir los comandos que entienden, en lugar de aprender a usar listas.

Los estudiantes de iGame se involucran en el análisis cuando juzgan si sus abstracciones fueron correctas y eficientes. El análisis de la corrección se centra en si produjeron el juego que pretendían, es decir, si el juego se juega de la manera que ellos quieren o si el juego que fue diseñado para ser divertido fue, de hecho, divertido. El análisis de la eficiencia implica la creación del código más simple para lograr el comportamiento deseado. El análisis se lleva a cabo durante el proceso de prueba y depuración de su juego, y los estudiantes a menudo necesitan un

motivador externo para enfocarse en la eficiencia porque cuando su juego está funcionando, ven pocas razones para editar el código. Los estudiantes también prueban los juegos de sus compañeros y los analizan en términos de jugabilidad y si han creado las mejores (más eficientes, más creíbles o más divertidas) abstracciones.

2.4 Otros dominios

Los dominios mencionados no tienen pretensión alguna de ser una lista completa. Hay muchos otros dominios, como el diseño y programación de páginas web, aplicaciones para teléfonos móviles, etc., que tienen potencial para desarrollar el pensamiento computacional en los jóvenes. Entre estos ejemplos, es común el uso activo de conceptos clave de pensamiento computacional: abstracción, automatización y, en diversos grados, análisis, por parte de los jóvenes dentro de los programas de la escuela secundaria. A través de estos ejemplos, postulamos que el PeCo no solo es posible en el nivel de la escuela secundaria, sino que puede integrarse fácilmente en actividades que alientan a los jóvenes a ser creadores, innovadores y solucionadores de problemas. Los proyectos de pensamiento computacional como los ejemplificados, apoyan un ciclo iterativo de refinamiento que permite aumentar el sentido de realización, donde los estudiantes están capacitados para imaginar, crear, jugar, compartir y reflexionar sobre lo que están aprendiendo [9]. En todos estos proyectos, el resultado final es un producto único creado por los estudiantes.

3 APOYAR EL CRECIMIENTO DEL PENSAMIENTO COMPUTACIONAL

Basándonos en nuestras experiencias con los jóvenes que aprenden el pensamiento computacional tanto durante la jornada escolar como en contextos extraescolares, sugerimos pasos concretos que se pueden tomar para apoyar el desarrollo del pensamiento computacional.

3.1 Entornos computacionales ricos

El primero es el uso de entornos computacionales ricos. Los entornos computacionales ricos son aquellos en los que las

abstracciones y los mecanismos subyacentes se pueden inspeccionar, manipular y personalizar.

Por ejemplo, considere las diferencias entre SimCity y un modelo en StarLogo TNG. En SimCity, un usuario puede agregar edificios a una ciudad y ver las correlaciones entre la adición de edificios y la producción de CO₂, pero las fórmulas y el modelo subyacentes están ocultos a la vista.

Comparemos eso con un entorno StarLogo TNG en el que el usuario puede "mirar debajo del capó" e inspeccionar las relaciones causales y abstracciones que están embebidas en un modelo. El rico entorno computacional es aquel en el que el usuario puede desarrollar habilidades de pensamiento computacional y transformarse de usuario en creador. Ver la Figura 4.



Figura 4: Inspección del mecanismo de infección en un modelo de contagio básico en StarLogo TNG.

3.2 Progresión de tres etapas "Usar-Modificar-Crear"

En segundo lugar, proponemos usar una progresión de tres etapas para involucrar a los jóvenes en el PeCo dentro de estos ricos entornos computacionales. Esta progresión, denominada Usar-Modificar-Crear, describe un patrón de participación (ver Figura 5) que se consideró que apoyaba y profundizaba la adquisición del PeCo por parte de los jóvenes en los proyectos NSF de los autores. Se basa en la premisa de que el andamiaje de interacciones cada vez más profundas promoverá la adquisición y el desarrollo del pensamiento computacional. En la etapa de uso, los estudiantes son consumidores de la creación de otra persona. Por ejemplo, ejecutan experimentos utilizando modelos de computadora preexistentes, ejecutan un programa que controla un robot o juegan un juego de computadora listo para usar.

Con el tiempo comienzan a modificar el modelo, juego o programa con niveles crecientes de sofisticación. Por ejemplo, un estudiante puede

inicialmente querer cambiar el color de un personaje o algún otro atributo puramente visual.

Más tarde, es posible que el estudiante desee cambiar el comportamiento del personaje de una manera que implique el desarrollo de nuevas piezas de código. La modificación de este tipo requiere la comprensión de al menos un subconjunto de la abstracción y la automatización contenidas en un programa, modelo o juego.

A través de una serie de modificaciones y refinamientos iterativos, se desarrollan nuevas habilidades y conocimientos a medida que lo que alguna vez fue de otra persona se convierte en propio. A medida que los jóvenes adquieren habilidades y confianza, se les puede animar a desarrollar ideas para nuevos proyectos computacionales de su propio diseño que aborden los problemas de su elección. Dentro de esta etapa de "creación", entran en juego los tres aspectos clave del pensamiento computacional: abstracción, automatización y análisis.

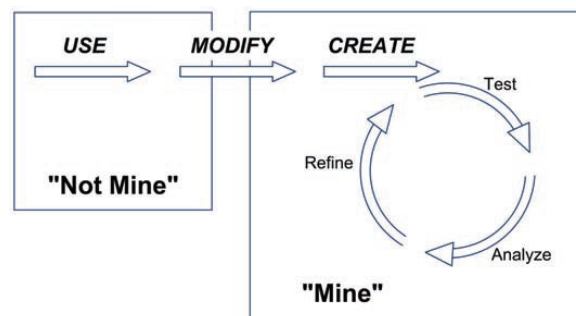


Figura 5: Progresión de aprendizaje Usar-Modificar-Crear

A través de esta progresión, es importante mantener un nivel de desafío que apoye el crecimiento y limite la ansiedad. Como señala Repenning [8], los estudiantes pueden mantener su sentido de flujo cognitivo [1] a medida que avanzan iterativamente a través de una serie de proyectos. En este trabajo, los estudiantes abordan desafíos de diseño cada vez más altos a medida que aumentan sus habilidades y capacidades. Las actividades que alguna vez fueron "demasiado difíciles" y que provocaron ansiedad se vuelven posibles con experiencias apropiadas y cada vez más desafiantes.

Por el contrario, el aburrimiento se impondrá si los desafíos no van a la par con las habilidades en crecimiento [8]. Si bien abogamos por el uso de esta progresión de tres etapas para fomentar el crecimiento a lo largo del tiempo y con una capacidad creciente, también planteamos una

advertencia acerca de tomarlo demasiado literalmente. Así como un joven adolescente pasa de la niñez a la adolescencia a los tumbos, no hay puntos de ruptura claros desde el uso hasta la modificación y la creación. Los jóvenes pueden pasar de usuarios a modificadores y a creadores.

3.3 Otros dominios

Los ejemplos de pensamiento computacional en programas para jóvenes descritos hasta ahora tenían lugar después de finalizado el horario escolar, ya sea durante los fines de semana, vacaciones y/o durante el verano. EcoScienceWorks (ESW) es un programa en Maine, Estados Unidos, que aprovecha la iniciativa de computadora portátil uno a uno del estado para involucrar a los estudiantes con simulaciones ambientales como parte del plan de estudios de ciencias de la jornada escolar. Este proyecto también expone a los estudiantes a desafíos simples de programación como una forma de presentarles el pensamiento computacional que subyace a las simulaciones. A través de la experimentación guiada, EcoScienceWorks profundiza la comprensión de los estudiantes tanto

computacional eran lo que los estudiantes tenían que hacer para cumplir con las metas explícitas de aprendizaje del contenido. Es decir, un plan de estudios de ecología que supuestamente requería PeCo fue diseñado para reemplazar el plan de estudios existente que se enfocaba en la transferencia de contenido. Con este diseño pedagógico, los conceptos básicos de ecología requeridos podrían cubrirse con mucha mayor profundidad, y se fomentaba el PeCo mediante el uso y la comprensión de modelos computacionales.

Si bien este trabajo ofrece un ejemplo prometedor, es importante reconocer los recursos que fueron necesarios para que tuviera éxito.

La infraestructura no fue un obstáculo significativo ya que cada estudiante tenía acceso a una computadora portátil, se había establecido el apoyo del distrito y el personal del proyecto brindó un apoyo intensivo en forma de desarrollo profesional y asistencia continua. Las aplicaciones transformadoras del pensamiento computacional pueden funcionar en las escuelas con todos estos ingredientes en su lugar. Implementar el pensamiento computacional durante la jornada escolar es una visión convincente, pero existen desafíos sustanciales para esto, incluidos los

Implementar el pensamiento computacional durante la jornada escolar es una visión convincente, pero existen desafíos sustanciales para esto, incluidos los estándares curriculares existentes, la falta de oportunidades para que los docentes aprendan el pensamiento computacional como parte de su desarrollo profesional y la falta de acceso a la infraestructura necesaria.

estándares curriculares existentes, la falta de oportunidades para que los docentes aprendan el pensamiento computacional como parte de su desarrollo profesional y la falta de acceso a la infraestructura necesaria. En consecuencia, gran parte del trabajo sobre pensamiento computacional con jóvenes permanece en entornos extraescolares.

de la ecología como del modelado por computadora.

El éxito del proyecto ha sido en parte el resultado de abordar algunos de los desafíos en la introducción frontal del pensamiento computacional en el aula. Por ejemplo, debido a que el pensamiento computacional no se evalúa mediante pruebas estandarizadas, es difícil en el clima educativo actual para los profesores enseñar directamente los conceptos del pensamiento computacional.

El personal de ESW abordó esta restricción mediante el diseño de un plan de estudios de ecología basado en simulación en el que las partes del plan de estudios de pensamiento

Como se muestra en este documento, están surgiendo nuevas oportunidades para fomentar el pensamiento computacional, y los programas financiados por NSF están explorando activamente formas en las que el pensamiento computacional funciona tanto en entornos escolares como extraescolares.

4 CONCLUSIONES

El llamado para integrar el PeCo en entornos escolares se ha vuelto cada vez más fuerte, a pesar de la falta de descripciones de cómo se ve realmente aprender a pensar computacionalmente entre los jóvenes. En este

documento, hemos contribuido a los esfuerzos para definir y apoyar el PeCo para los jóvenes mediante el uso de ejemplos de varios proyectos para señalar dos puntos clave.

El primer punto clave es que las definiciones existentes de pensamiento computacional se pueden aplicar a entornos escolares. Los ejemplos muestran que los jóvenes pueden participar en aspectos clave del pensamiento computacional dentro de programas que se enfocan en modelización y simulación, robótica y diseño y desarrollo de juegos. Los estudiantes de aulas heterogéneas pueden utilizar la abstracción, la automatización y el análisis para crear productos originales cuando se les da acceso a entornos de aprendizaje ricos que incluyen docentes capacitados, consideraciones de desarrollo y, por lo general, incluyen nueva tecnología. Sin embargo, el campo requiere procedimientos de evaluación sistemáticos que se basen en la investigación existente de las ciencias del aprendizaje para describir la progresión del desarrollo de estos tres constructos del PeCo. Algunos de los autores están probando actualmente una variedad de enfoques de evaluación.

El segundo punto clave es que el pensamiento computacional tiene lugar en un continuo. La progresión de usar-modificar-crear se ofrece como un marco para educadores e investigadores que buscan cómo se desarrolla el PeCo y cómo se puede apoyar ese desarrollo. Pero se necesita investigación para comprender por qué los estudiantes piensan en diferentes niveles de abstracción, automatización y análisis. Estas diferencias pueden ser una función de los estudiantes que trabajan en diferentes fases de la progresión de aprendizaje de usar-modificar-crear.

Por ejemplo, sugerimos que pasar de modificar a crear un proyecto original requiere niveles crecientes de representación y comprensión abstractas. De manera similar, el análisis simple incluye probar y depurar un programa, mientras que un nivel más profundo de análisis implicaría tratar de determinar si un modelo se puede validar con datos del mundo real. Como base para avanzar, el marco de uso, modificación y creación ofrece una progresión útil para desarrollar el pensamiento computacional a lo largo del tiempo. Su mayor beneficio es ilustrar los beneficios que surgen de involucrar a los jóvenes

en tareas cada vez más complejas y darles una mayor autonomía en su aprendizaje.

Este documento tiene como objetivo informar los esfuerzos para involucrar a los estudiantes de nivel escolar en el pensamiento computacional y evaluar el valor de estos esfuerzos. Recomendamos que los esfuerzos futuros sean más específicos sobre el tipo y nivel de pensamiento computacional que se abordará. El proyecto CS Principles ha hecho avanzar este esfuerzo en el contexto de las clases de informática de la escuela secundaria, y una publicación reciente de Snyder [10] describe prácticas específicas de pensamiento computacional.

Este documento se basa en los esfuerzos existentes para describir el alcance y la naturaleza del PeCo [7], así como los conceptos involucrados en el PeCo y cómo los jóvenes deberían poder usar esos conceptos [2]. Esperamos que este documento contribuya a un diálogo sobre las dimensiones más importantes del PeCo en el nivel escolar, cómo se desarrollan los diferentes aspectos del PeCo, el papel del contexto y la motivación en este desarrollo, y estrategias efectivas para involucrar a los jóvenes en el pensamiento computacional.

Agradecimientos

Partes de este documento fueron adaptadas de Computational Thinking for Youth, ITEST Working Group on Computational Thinking (2010) con permiso del Education Development Center, Inc., Newton, Massachusetts.

Referencias

- [1] Csikszentmihalyi, M. (1990). Flow: la psicología de la experiencia óptima. Nueva York: Harper.
- [2] Reunión de líderes de pensamiento de pensamiento computacional. (2010). Asociación de Profesores de Ciencias de la Computación y Sociedad Internacional para la Tecnología en la Educación. 18-19 de abril de 2010.
- [3] Cuny, J., Snyder, L. y Wing, J. (2010). Pensamiento computacional: una definición. (en prensa)
- [4] Henderson, PB (2009). Pensamiento computacional ubicuo. Computadora, 42 (10), 100-102. Disponible en línea en <http://www.computer.org/portal/web/csdl/doi/10.1109/MC.2009.334>. Consultado el 29 de junio de 2010.
- [5] Lu, JJ & Fletcher, GHL (2009). Pensando en el pensamiento computacional. Conferencia del Grupo de Interés Especial sobre Educación en Ciencias de la Computación de ACM, (SIGCSE 2009), (Chattanooga, TN, EE. UU.), ACM Press.

Disponible en línea en <http://portal.acm.org/citation.cfm?id=1508959&dl=ACM&coll=portal>. Consultado el 29 de junio de 2010.

[6] Moursund, D. (2009). Pensamiento computacional. IAE-pedia. Disponible en línea en http://iaepedia.org/Computational_Thinking.

Consultado el 8 de agosto de 2010.

[7] Academias Nacionales de Ciencias. (2010). Informe de un taller sobre el alcance y la naturaleza del pensamiento computacional. Washington DC: Prensa de las Academias Nacionales.

[8] Repenning, A. y Ioannidou, A. (2008). Ampliación de la participación a través del diseño escalable de juegos, ACM Special Interest Group on Computer Science Education Conference, (SIGCSE 2008), (Portland, Oregon USA), ACM Press. Disponible en línea en <http://www.cs.colorado.edu/~ralex/papers/index.html>, consultado el 19 de abril de 2010.

[9] Resnick, M. (2007). Todo lo que realmente necesito saber (sobre el pensamiento creativo) lo aprendí (al estudiar cómo aprenden los niños) en el jardín de infantes. Conferencia ACM Creativity & Cognition, Washington DC, junio de 2007. Disponible en línea en <http://web.media.mit.edu/~mres/papers.html>, consultado el 19 de abril de 2010.

[10] Snyder, L. (2010). Siete prácticas de pensamiento computacional. Disponible en línea en <http://csprinciples.cs.washington.edu/sevenpractices.html>, consultado el 2 de agosto de 2010.

[11] Wing, J. (2006). Pensamiento computacional. Comunicaciones del ACM 49 (3), 33-35.

IRENE LEE

Santa Fe Institute, 1399 Hyde Park Road, Santa Fe, New Mexico 87505 USA

lee@santafe.edu

FRED MARTIN

University of Massachusetts Lowell, Olsen Hall Rm 208, Lowell, Massachusetts 01854 USA

fredm@cs.uml.edu

JILL DENNER

ETR Associates, 4 Carbonero Way, Scotts Valley, California 95066 USA

jilld@etr.org

BOB COULTER

Missouri Botanical Garden, PO Box 299, St. Louis, Missouri 63166-0299 USA

bob.coulter@mobot.org

WALTER ALLAN

Foundation for Blood Research

ScienceWorks for ME

8 Science Park Road, Scarborough, Maine 04070 USA

allan@fbr.org

JERI ERICKSON

Foundation for Blood Research, P.O. Box 190

8 Science Park Road, Scarborough, Maine 04070-0190 USA

jerickso@maine.rr.com

JOYCE MALIN-SMITH

ITEST Learning Resource Center, Education Development Center

55 Chapel Street, Newton Massachusetts 02458-1060 USA

jmsmith@edc.org

LINDA WERNER

University of California, Santa Cruz, Baskin Engineering

1156 High Street, Santa Cruz, California 95064 USA

linda@soe.ucsc.edu